

Phương pháp rời rạc hóa liên quan đến đường tròn

Gao Yihan

Trường Phổ thông trực thuộc Đại học Thanh Hoa

Nguồn gốc: Discretization methods related to circles

IOI China candidate team papers / 2007 / day 1 / Gao Yihan

Abstract

Trong các bài toán hình học tính toán, rời rạc hóa là một phương pháp khá tổng quát. Khi giải những bài toán liên quan đến các hình tạo bởi đường thẳng như hình chữ nhật, nó có thể giảm mạnh độ phức tạp thời gian và bộ nhớ. Nhưng khi bài toán liên quan đến đường tròn, cách rời rạc hóa trực tiếp gặp nhiều khó khăn. Bài viết này bàn về cách dùng rời rạc hóa trong lớp bài toán đó, rồi qua một vài ví dụ minh họa cách dùng rời rạc hóa để giải các bài toán hình học tính toán liên quan đến đường tròn.

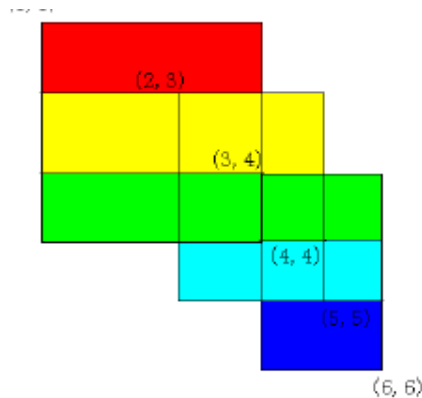
Từ khóa. Hình học tính toán; đường tròn; rời rạc hóa.

1 Dẫn nhập

Với phần lớn thuật toán, dữ liệu liên tục không phải là đối tượng xử lý thuận tiện; thường phải rời rạc hóa chúng trước khi xử lý. Một phương pháp rời rạc hóa hiệu quả có thể làm giảm độ phức tạp của bài toán, vì vậy cách thiết kế thuật toán rời rạc hóa là một chủ đề có ý nghĩa. Bài viết này khảo sát các hình thức ứng dụng của rời rạc hóa thông qua phân tích các bài toán hình học tính toán liên quan đến đường tròn.

Trước hết ta xem rời rạc hóa giải bài toán tính diện tích hợp các hình chữ nhật như thế nào.

Trên mặt phẳng có ba hình chữ nhật, lần lượt là $(1, 1) - (4, 4)$, $(2, 3) - (5, 5)$, $(3, 4) - (6, 6)$. Cần tính diện tích hợp của ba hình chữ nhật này.

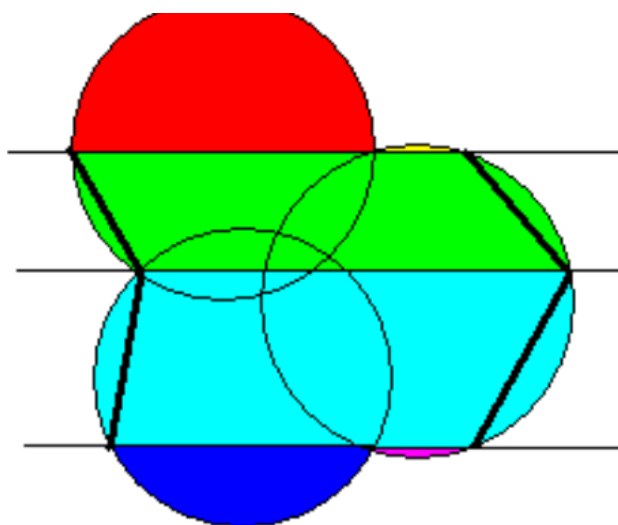


Hình 1: Ba hình chữ nhật chồng lẫn nhau; các tung độ biên chia mặt phẳng thành các dải ngang.

Ta dùng rời rạc hóa để chia trục tung của mặt phẳng liên tục thành năm khoảng: $(1, 2)$, $(2, 3)$, $(3, 4)$, $(4, 5)$, $(5, 6)$. Có thể thấy rằng trong cùng một khoảng, thuộc tính là giống nhau: trên trục hoành, phần bị các hình chữ nhật phủ là như nhau. Vì thế diện tích của mỗi vùng có thể tính nhanh bằng phép nhân, rồi cộng diện tích của mọi khoảng lại. Từ đó thấy được ưu thế của rời rạc hóa trong việc giảm độ phức tạp.

2 Phương pháp

Khi đối tượng tính toán không phải hình chữ nhật mà là đường tròn, chẳng hạn tính diện tích hợp của ba hình tròn, vì biên của hình tròn là đường cong và không tồn tại các vùng chữ nhật cố định, thoát nhìn đường như không thể dùng cách rời rạc hóa ở trên để chia thành các vùng tương tự. Nhưng nếu chia trục tung thành một số khoảng thích hợp, toàn bộ hình được chia thành các phần; mỗi phần lại được cấu thành từ những mảnh đơn giản như cung tròn và hình thang, tạo nên những vùng rời rạc có thuộc tính kết hợp. Cách đó vẫn làm bài toán đơn giản hơn.



Hình 2: Ba hình tròn được cắt bởi các đường ngang qua những tung độ quan trọng; mỗi dải chỉ chứa những mảnh đơn giản.

Điều này gợi ý rằng ta có thể cắt hình thành một số khối tương đối đơn giản, sao cho diện tích của từng phần đều dễ tính. Đó là ứng dụng cơ bản nhất của rời rạc hóa trong hình học tính toán liên quan đến đường tròn.

Các bước chung của thuật toán:

1. Dựa vào bài toán, chia các hình tròn trong mặt phẳng thành một số vùng, sao cho mỗi vùng có những thuộc tính thô tương đối đơn giản. Thông thường có thể dùng đường thẳng để chia.
2. Dựa vào thuộc tính đó, xác định thuật toán cụ thể cho các hình tròn trong vùng và tính kết quả của từng vùng.
3. Tổng hợp kết quả của mọi vùng để thu được đáp án cuối cùng.

3 Ví dụ ứng dụng và kết quả

Sau đây ta phân tích cụ thể hai ví dụ để bàn về hình thức ứng dụng của thuật toán.

3.1 Ví dụ 1: Dolphin Pool

Tóm tắt đề bài

Trên mặt phẳng có n đường tròn. Không có hai đường tròn nào tiếp xúc, và không có tâm của đường tròn nào nằm bên trong một đường tròn khác. Hãy tính có bao nhiêu miền đóng không bị bất cứ hình tròn nào phủ.

Input. Dòng đầu chứa một số n ($0 < n \leq 20$), là số đường tròn. Tiếp theo là n dòng, mỗi dòng nhập ba số nguyên: tọa độ tâm x, y và bán kính r , với $1 \leq x, y \leq 1000$, $1 \leq r \leq 100$.

Output. In ra một số nguyên, là số miền đóng.

Phân tích ban đầu

Dù đề bài đã có một số ràng buộc về vị trí các đường tròn, thực tế các khả năng vẫn rất nhiều. Ta hy vọng tìm được một thuật toán đơn giản và tổng quát.

Vì dữ liệu đều là số nguyên, ý tưởng đầu tiên là yêu cầu độ chính xác của bài này không cao, nên có thể dùng một thuật toán dựa trên flood fill:

Trong mặt phẳng, cứ cách một khoảng cố định lại lấy một điểm để tạo thành lưới điểm; xóa mọi điểm nằm trong các hình tròn; sau đó flood fill trên các điểm còn lại để tìm số thành phần liên thông; cuối cùng trừ đi 1 là đáp án. Độ phức tạp của thuật toán là $\mathcal{O}(k^2n)$, trong đó k là số điểm lấy trên độ dài 1000.



Hình 3: Lưới điểm phủ lên các đường tròn; các điểm nằm trong hình tròn bị loại bỏ rồi flood fill trên phần còn lại.

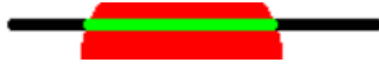
Ta biết rằng tính đúng đắn hoặc độ chính xác của thuật toán này phụ thuộc vào kích thước k . Với giới hạn thời gian của bài, thuật toán này còn rất xa mới đạt yêu cầu, nên cần cải tiến.

Bước rời rạc hóa thứ nhất

Xét từng hàng ngang của tập điểm, ta nhận thấy mỗi hình tròn phủ trên hàng đó một đoạn liên tục.

Vì thế, trên mỗi hàng ngang, không cần liệt kê từng điểm; thay vào đó ta tính đoạn mà mỗi hình tròn phủ trên hàng ấy. Bước này có thể tính bằng công thức toán trong thời gian hằng số:

$$\begin{aligned} Left &= circle.x - \sqrt{circle.radius^2 - (circle.y - line.height)^2}, \\ Right &= circle.x + \sqrt{circle.radius^2 - (circle.y - line.height)^2}. \end{aligned}$$



Hình 4: Giao của một đường ngang với một hình tròn là một đoạn liên tục, nên không cần xét từng điểm trên hàng.

Sau đó gộp các đoạn giao nhau. Trong bước tiền xử lý, có thể sắp xếp các hình tròn theo hoành độ, nhờ vậy việc gộp đoạn sẽ đơn giản hơn.

Cuối cùng, dựa vào các đoạn bị phủ, ta tìm những đoạn không bị hình tròn nào phủ.

Mô phỏng cách thứ nhất, ta ghi lại toàn bộ các đoạn, nối một cạnh giữa hai đoạn ở hai hàng kề nhau nếu chúng có phần chung, rồi tính số thành phần liên thông; cuối cùng trừ đi 1 là đáp án.

Vì số đoạn quá lớn, dùng DFS hay BFS đều tốn quá nhiều bộ nhớ. Ta có thể lợi dụng tính đặc biệt của đồ thị để tạo ra một cách quét tuần tự hiệu quả hơn nhằm đếm số thành phần liên thông:

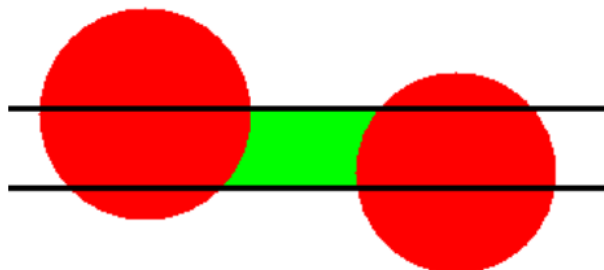
1. Khởi tạo hàng ngang đầu tiên. Hàng này chỉ có một đoạn, thuộc về một tập hợp.
2. Với mỗi đoạn của hàng ngang kế tiếp, khởi tạo một tập hợp mới chỉ chứa đoạn đó.
3. Nếu hai đoạn thuộc hai hàng có giao nhau, gộp hai tập hợp chứa chúng.
4. Nếu mọi đoạn trong một tập hợp đều không có hậu duệ ở hàng kế tiếp, tăng đáp án lên 1.
5. Lặp lại các bước 2–4 cho đến khi quét hết hàng cuối cùng.

Có thể thấy thuật toán này có độ phức tạp $\mathcal{O}(nk)$, và với giới hạn thời gian nó có thể đạt độ chính xác khá cao.

Tuy vậy, trong đề bài tồn tại một số dữ liệu đòi hỏi độ chính xác rất cao, còn bản thân thuật toán này không đủ chính xác, nên cuối cùng vẫn bị Wrong Answer. Vì vậy ta vẫn cần cải tiến thêm.

Bước rời rạc hóa thứ hai

Quan sát kỹ chương trình, ta phát hiện dường như có rất nhiều công việc vô ích. Xét ví dụ sau.



Hình 5: Trong một vùng không có đường tròn nào khác, quan hệ kế thừa giữa các đoạn giữ nguyên qua rất nhiều hàng quét.

Trong vùng được đánh dấu, trực giác cho thấy đó chắc chắn là cùng một miền, vì bên trong hoàn toàn không có hình tròn nào khác. Nhưng chương trình vẫn vất vả theo dõi từng bước, cho đến cuối cùng mới xác nhận hàng dưới cùng đúng là hậu duệ của hàng trên cùng.

Từ đó ta có một ý tưởng khác: bỏ qua các thao tác dư thừa. Ta nhận thấy quan hệ kế thừa giữa các đoạn hầu như không thay đổi, chỉ thay đổi tại một số ít thời điểm:

- Khi một hình tròn mới xuất hiện, một đoạn bị chia làm hai nửa.
- Khi hai hình tròn cắt nhau, một đoạn dần thu nhỏ rồi bị nuốt mất.
- Khi hai hình tròn cắt nhau, một đoạn mới được sinh ra.
- Ở điểm thấp nhất của một hình tròn, hai đoạn gộp thành một đoạn.

Như vậy, ta chỉ cần ghi lại toàn bộ các sự kiện điểm, mô phỏng sự sinh trưởng và tiêu vong của các đoạn, rồi giải bài toán. Chú ý rằng ta không cần ghi lại độ dài chính xác của từng đoạn.

Tuy nhiên, vì sự kiện điểm có bốn kiểu định tính, trong đó hai kiểu ở giữa khó phân biệt, chương trình rất dễ sai. Thực ra thậm chí không cần ghi lại tính chất của sự kiện điểm. Với mỗi sự kiện điểm, lấy một khoảng thời gian cực ngắn ngay trước và ngay sau nó, rồi dùng cách quét tuần tự ở phương pháp thứ hai: tiền nhiệm của sự kiện điểm tiếp theo trực tiếp kế thừa hậu duệ của sự kiện điểm hiện tại.

Đến đây, nhờ dùng rời rạc hóa, bài toán đã được giải quyết hoàn chỉnh. Độ phức tạp thời gian là $\mathcal{O}(n^3)$.

3.2 Ví dụ 2: Empire strikes back

Tóm tắt đề bài

Đất nước của Saddam là một hình tròn bán kính R , tâm tại $(0, 0)$. Trong đó có N nhà máy vũ khí hóa học, tọa độ lần lượt là (X_i, Y_i) . George II sẽ thả bom xuống từng nhà máy; mỗi quả bom san phẳng mọi thứ trong một hình tròn có tâm là nhà máy và bán kính nhất định. Vì không biết Saddam đang ẩn náu ở đâu, George II muốn bán kính sát thương của các quả bom phủ kín toàn bộ lãnh thổ. Hỏi bán kính sát thương nhỏ nhất cần dùng là bao nhiêu.

Input. Dòng đầu chứa hai số N ($1 \leq N \leq 300$) và R ($1 \leq R \leq 1000$), lần lượt là số nhà máy và bán kính lãnh thổ. Tiếp theo là N dòng, mỗi dòng chứa mô tả của một nhà máy bằng hai số (X_i, Y_i) , thỏa $X_i^2 + Y_i^2 \leq R^2$.

Output. In ra bán kính nhỏ nhất, giữ 5 chữ số sau dấu thập phân.

Ý tưởng ban đầu

Đề bài chỉ yêu cầu một số, nên ta xét dùng tìm kiếm nhị phân để tìm đáp án. Các bước nhị phân như sau:

1. Khởi tạo $Left = 0$, $Right = 2R$.
2. Tính số ở giữa:

$$Mid = (Left + Right)/2.$$
3. Kiểm tra với bán kính Mid có phủ kín hình tròn lớn hay không. Nếu có thì đặt $Right = Mid$, nếu không thì đặt $Left = Mid$.
4. Khi $Right - Left < 10^{-6}$ thì dừng; nếu chưa thì lặp lại bước 2–3.

Tìm kiếm nhị phân chuyển bài toán tối ưu thành bài toán khả thi. Trọng tâm còn lại là xác định, với một bán kính đã cho, hình tròn lớn có bị phủ kín hay không.

Phân tích thuật toán

Thoạt nhìn, bài này rất giống ví dụ trước. Nhưng nếu dùng trực tiếp thuật toán của ví dụ trước thì thời gian hoàn toàn không chịu nổi.

Phân tích kỹ hơn mới thấy hai bài có một chút khác biệt: ví dụ trước yêu cầu tính số miền, còn bài này chỉ cần phán định có phủ kín hay không. Bài toán đã chuyển từ đếm sang phán định, trong khi ta vẫn dùng tư duy cũ để giải, nên không thể có thuật toán hiệu quả. Vì vậy cần thiết kể lại thuật toán.

Xét ý tưởng nguyên thủy nhất: liệt kê mọi điểm trong hình tròn lớn, rồi kiểm tra điểm đó có được hình tròn nhỏ nào phủ hay không. Nếu mọi điểm trong hình tròn lớn đều được hình tròn nhỏ phủ, thì hình tròn lớn chắc chắn được phủ kín.

Tuy nhiên, vì đề bài yêu cầu độ chính xác rất cao, thuật toán này chắc chắn không thể cho đáp án đúng. Ta nghĩ đến việc dùng rời rạc hóa để tối ưu thuật toán đó.

Bước rời rạc hóa thứ nhất

Tiếp tục dùng tư tưởng của ví dụ trước: coi một hàng ngang là một chỉnh thể. Với mỗi hàng ngang, tính đoạn được mỗi hình tròn nhỏ phủ, rồi gộp tất cả các đoạn giao nhau. Nếu cuối cùng chỉ còn một đoạn và đoạn đó chứa đoạn mà hình tròn lớn phủ trên đường ngang ấy, ta có thể kết luận rằng hình tròn lớn đã được phủ kín trên đường ngang đó. Nếu trên mọi đường ngang hình tròn lớn đều được phủ kín, ta có thể xấp xỉ xem hình tròn lớn đã được các hình tròn nhỏ phủ kín.

Vì bài toán chỉ cần phán định có phủ kín hay không, chọn các điểm đặc biệt là một ý tưởng tốt. Với mỗi hàng ngang, xét các điểm cực gần hai phía trái và phải của các điểm then chốt, rồi kiểm tra chúng có được các hình tròn khác phủ hay không.



Hình 6: Các điểm đặc biệt nằm rất gần hai bên của những giao điểm then chốt trên mỗi hàng ngang.

Có thể thấy rằng, nếu chỉ xét tất cả điểm đặc biệt đã chọn, hình tạo bởi các điểm ấy đúng bằng hợp của các hình thu được khi dịch mỗi hình tròn sang trái một chút và sang phải một chút. Phần nằm bên trong hình tròn ban đầu có thể bỏ qua. Vì lượng dịch rất nhỏ, ta có thể xem mọi điểm đặc biệt đều nằm trên biên của hình tròn trước khi dịch.

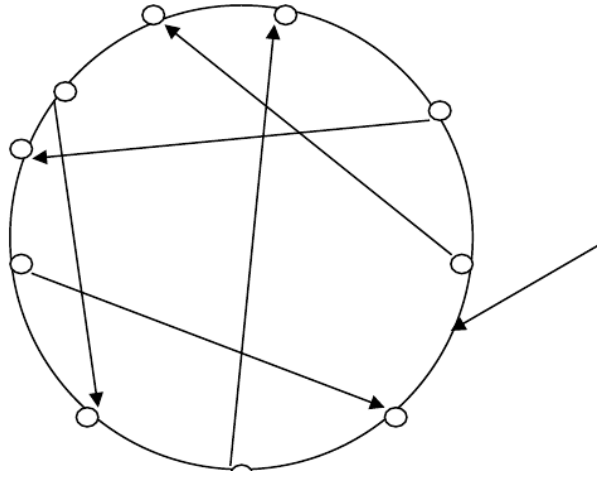
Bước rời rạc hóa thứ hai

Theo nguyên tắc xử lý tập trung, ta xử lý đồng thời mọi điểm trên cùng một đường tròn. Ta thấy rằng phần một hình tròn phủ trên biên của một hình tròn khác luôn là một khoảng liên tục. Vì vậy bài toán chuyển thành: nhiều khoảng nhỏ có phủ kín toàn bộ một khoảng hay không. Bài toán này rất giống bài thu được sau bước rời rạc hóa thứ nhất trong ví dụ 1, nên có thể giải bằng thuật toán tương tự.

Phân tích độ phức tạp

Độ phức tạp của nhị phân là $\mathcal{O}(\log R)$. Sắp xếp các khoảng tốn $\mathcal{O}(N \log N)$. Gộp các khoảng tốn $\mathcal{O}(N)$. Liệt kê N đường tròn tốn $\mathcal{O}(N)$. Vì vậy tổng độ phức tạp của thuật toán là

$$\mathcal{O}(N^2 \log N \log R).$$



Hình 7: Trên biên của một hình tròn, các hình tròn khác phủ các cung liên tục; nếu còn một cung không bị phủ thì bán kính hiện tại chưa đủ.

Đến đây, bài toán đã được giải quyết hoàn chỉnh.

Một vài tối ưu nhỏ

Thực ra, nếu khi sắp xếp ta sắp theo góc cực của trung điểm khoảng, thì thứ tự các khoảng không phụ thuộc vào độ dài bán kính và có thể được hoàn thành trong tiền xử lý. Khi đó độ phức tạp giảm xuống

$$\mathcal{O}(N^2(\log N + \log R)).$$

Vì bán kính nhỏ nhất cần để phủ mỗi đường tròn là độc lập với nhau, ta có thể xem nó là một hằng số; điều cần tìm là giá trị lớn nhất trong các hằng số này. Do đó khi liệt kê từng đường tròn, trước hết kiểm tra xem nó có được phủ kín dưới bán kính nhỏ nhất hiện tại hay không. Nếu đã được phủ kín thì không cần nhị phân nữa. Yang Yi và Yuan Xinhao đã chứng minh độ phức tạp kỳ vọng của thuật toán này là

$$\mathcal{O}(N^2 \log N + N \log N \log R).$$

Chứng minh như sau. Xét thuật toán:

1. Chọn ngẫu nhiên một hình tròn nhỏ.
2. Dùng nhị phân tính bán kính nhỏ nhất cần để phủ kín nó.
3. So sánh với mọi hình tròn nhỏ còn lại.
4. Giữ lại những hình tròn nhỏ chưa được phủ kín.
5. Đệ quy.

Có thể thấy thuật toán này về bản chất giống thuật toán đã nêu ở trên. Thuật toán này thực ra là một dạng đặc biệt của quickselect, nên để chứng minh độ phức tạp của nó là $\mathcal{O}(N + \log N \log R)$. Vì vậy độ phức tạp của thuật toán gốc là $\mathcal{O}(N^2 \log N + N \log N \log R)$. Ngoài ra, có thể dùng ngẫu nhiên hóa để tránh trường hợp xấu nhất.

4 Tổng kết

Rời rạc hóa là một thuật toán khá phổ biến trong hình học tính toán. Khi bài toán liên quan đến đường tròn, rời rạc hóa có thể dùng việc rời rạc hóa vùng để chuyển bài toán thành một bài toán hình học tính

toán với các “hình” tương đối đơn giản. Bản chất của rời rạc hóa nằm ở việc biến “lẻ” thành “nguyên khối”, biến “cong” thành “thẳng”, từ đó đạt mục tiêu đơn giản hóa độ phức tạp thuật toán. Dĩ nhiên, vì quá trình này không tránh khỏi phá vỡ một số tính chất của đường cong, nên với tư cách là một phương pháp xử lý khá mộc mạc, rời rạc hóa thường không hiệu quả bằng những thuật toán trực tiếp khai thác tính chất của đường cong. Nhưng tính phổ dụng và sự giản dị của nó vẫn giúp nó có vị trí trong các thuật toán hình học tính toán. Tuy bài viết phân tích các bài toán hình học tính toán về đường tròn, các ý tưởng này rất dễ mở rộng sang những bài toán hình học tính toán về các đường cong khác.

Lời cảm ơn

Cảm ơn Yang Yi từ Trường Phổ thông trực thuộc Đại học Sư phạm An Huy và Yuan Xinhao từ Trường Trung học Changjun, Trường Sa, vì chứng minh độ phức tạp của thuật toán trong ví dụ 2.

Cảm ơn tất cả những người đã giúp đỡ tôi.

Tài liệu tham khảo

Rujia Liu, Liang Huang, *The Art of Algorithms and Informatics Olympiad*, Tsinghua University Press.

A Phụ lục

A.1 Chương trình nguồn cho ví dụ 1

Tệp chương trình nguồn của bài *Dolphin Pool*: `Dolphin Pool.pas`.

Tìm giao điểm của hai đường tròn. Đoạn mã sau dựa trên định lý cosin và có sai số nhất định.

```
int get_inter( int x1, int y1, int x2, int y2,
              double rad1, double rad2,
              double &ret1, double &ret2 )
{
    double b = sqrt( (double)( sqr( y2 - y1 ) + sqr( x2 - x1 ) ) );
    if ( b > rad1 + rad2 || b < rad1 - rad2 )
    {
        ret1 = 0;
        ret2 = 0;
        return 0;
    }
    else if ( b < rad2 - rad1 )
    {
        ret1 = 0;
        ret2 = 2 * pi;
        return 0;
    }
    // Các dòng trên xử lý những trường hợp đặc biệt.
    double mid = atan2( y2 - y1, x2 - x1 );
    double a = rad1, c = rad2;
```



```

double t = acos( ( sqr(a) + sqr(b) - sqr(c) ) / ( 2 * a * b ) );
// Ba dinh cua tam giac la hai tam duong tron va mot giao diem.
ret1 = mid - t;
ret2 = mid + t;
// Gia tri tra ve la goc cuc cua giao diem tren duong tron thu nhat.
return 0;
}

```

A.2 Nguyên đề ví dụ 1: Dolphin Pool

Giới hạn. Time Limit: 1000MS. Memory Limit: 10000K.

Description. Trong một hồ cá heo mới xây trên đảo Kish ở Vịnh Ba Tư, một trò chơi vui được tổ chức như sau. Người điều khiển trò chơi ném một số vòng nhựa xuống hồ sao cho tâm của không vòng nào nằm bên trong vòng khác, và không có hai vòng nào tiếp xúc nhau. Các cá heo được huấn luyện để, khi nghe còi của người điều khiển, nhảy lên qua những miền đóng hoàn toàn nằm ngoài các vòng, mỗi miền một cá heo. Chúng nhảy lên khi và chỉ khi số miền đóng đúng bằng số cá heo.

Hãy viết chương trình, theo mô tả input/output dưới đây, tìm số miền đóng giữa các vòng để giúp cá heo quyết định có nhảy hay không.

Input. Dòng đầu chứa số bộ test, không quá 20. Mỗi bộ test bắt đầu bằng một số nguyên N ($1 \leq N \leq 20$), là số vòng nhựa. Sau đó là N dòng, mỗi dòng chứa ba số nguyên: hai số đầu là tọa độ x, y của tâm đường tròn của vòng, số thứ ba là bán kính. Các tọa độ là số nguyên dương nhỏ hơn 1000, và bán kính nằm trong khoảng 1 ... 100.

Output. Với mỗi bộ test, in một dòng chứa số miền đóng của bộ test đó.

Sample Input.

```

2
4
100 100 20
100 135 20
135 100 20
135 135 20
1
10 10 40

```

Sample Output.

```

1
0

```

Source. Tehran 2000.

A.3 Nguyên đề ví dụ 2: Empire strikes back

Giới hạn. Time Limit: 1.0 second. Memory Limit: 16 MB.

Background. Nhiều năm đã trôi qua kể từ khi hoàng đế tốt bụng và thông thái George II Đại đế bắt đầu cai trị Đế quốc văn hóa và văn minh. Thế giới do ông tạo ra thật hùng mạnh và đẹp đẽ. Dưới sự cai trị của ông, những thành phố tráng lệ bằng cẩm thạch và thép vươn lên bầu trời, những cánh đồng rộng lớn được gieo hạt. Trẻ em vui chơi, người già cười nói, còn công nhân và nông dân cùng xây dựng lợi ích chung.

Nhưng một ngày, George biết rằng một hiểm họa đáng sợ đang đe dọa loài người. Nhà độc tài hiểm ác và tàn nhẫn Saddam III Khủng Khiếp, người cai trị một nước cộng hòa kém văn hóa và văn minh hơn nhiều, đang định chế tạo loại vũ khí hóa học mới nhất và giành quyền lực trên toàn hành tinh.

Problem. Theo báo cáo của cơ quan mật vụ, Saddam đã xây dựng N nhà máy vũ khí hóa học bên trong biên giới nước cộng hòa. Biên giới này là một hình tròn bán kính R có tâm tại điểm $(0, 0)$. Mỗi nhà máy được nguy trang khéo léo thành bệnh viện, trường học hoặc nhà dưỡng lão, và nằm tại điểm có tọa độ Descartes (X_i, Y_i) .

Ý đồ đe dọa của Saddam hoàn toàn không làm George hài lòng. Vì vậy ông quyết định phá hủy mọi nhà máy bằng ném bom. Tất cả bom phải có cùng bán kính sát thương hiệu dụng và được thả chính xác xuống nhà máy tương ứng.

Mỗi quả bom biến mọi vật trong bán kính sát thương hiệu dụng của nó thành một đám khí nóng bỏng. Chính sự thật này gợi cho George một ý nghĩ thú vị: thật tốt nếu có thể một công đôi việc và biến cả Saddam thành đám khí như vậy. Tiếc là cơ quan mật vụ không xác định được chính xác nơi ẩn náu của hắn. Vì thế George muốn tính bán kính sát thương hiệu dụng của bom sao cho, khi được thả chính xác xuống các nhà máy, chúng sẽ tiêu diệt Saddam bất kể hắn ở đâu trong nước cộng hòa. Mặt khác, chế tạo bom công suất lớn cần rất nhiều polonium-210 đắt tiền, nên bán kính sát thương hiệu dụng phải là nhỏ nhất.

Input. Dòng đầu chứa hai số nguyên N ($1 \leq N \leq 300$) và R ($1 \leq R \leq 1000$). Mỗi dòng trong N dòng tiếp theo chứa hai số nguyên X_i và Y_i , với $X_i^2 + Y_i^2 \leq R^2$, ứng với một nhà máy.

Output. In bán kính sát thương hiệu dụng cần tìm. Bán kính phải được in với ít nhất năm chữ số sau dấu thập phân.

Sample Input.

```
4 4
0 2
0 -2
2 0
-2 0
```

Sample Output.

```
2.94725152
```